Chapter 6

Putting Your Skills to the Test: Level 1 Challenges (Basic Concepts)

Section 1: Numbers and Strings

In this section, we'll explore challenges 1-10, which focus on working with numbers, strings, and user input in Python. These challenges are designed to help beginners understand the basic concepts of handling numerical data, manipulating strings, and interacting with users through input/output operations.

Challenge 1: Sum of Two Numbers

Create a Python script that asks the user to input two numbers and computes their total.

```python
# Challenge 1: Sum of Two Numbers
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

sum = num1 + num2
print("Sum:", sum)
```

This program requests the user to input two numbers, converts them into floating-point numbers, computes their sum, and subsequently displays the result.

Challenge 2: Area of a Rectangle

Write a Python program that calculates the area of a rectangle given its length and width.

```python
# Challenge 2: Area of a Rectangle
length = float(input("Enter the length of the rectangle: "))
width = float(input("Enter the width of the rectangle: "))

area = length * width
print("Area of the rectangle:", area)
```

This program prompts the user to enter the length and width of a rectangle, calculates its area, and then prints the result.

Challenge 3: Volume of a Cylinder

Write a Python program that calculates the volume of a cylinder given its radius and height.

```python
# Challenge 3: Volume of a Cylinder
import math

radius = float(input("Please input the cylinder's radius: "))
height = float(input("Enter the height of the cylinder: "))
```

The volume is calculated as the product of π, the square of the radius, and the height.

```
print("Volume of the cylinder:", volume)
```

This program prompts the user to enter the radius and height of a cylinder, calculates its volume using the formula $\pi r^2 h$, and then prints the result.

Challenge 4: String Concatenation

Write a Python program that prompts the user to enter two strings and concatenates them together.

```python
# Challenge 4: String Concatenation
string1 = input("Enter the first string: ")
string2 = input("Enter the second string: ")

concatenated_string = string1 + string2
print("Concatenated string:", concatenated_string)
```

This program prompts the user to enter two strings, concatenates them together, and then prints the result.

Challenge 5: Reverse a String

Write a Python program that prompts the user to enter a string and then prints the reverse of that string.

```python
# Challenge 5: Reverse a String
string = input("Enter a string: ")
```

```
reversed_string = string[::-1]
print("Reversed string:", reversed_string)
```

This program prompts the user to enter a string, reverses the string using slicing, and then prints the result.

Challenge 6: Count Vowels in a String

Write a Python program that prompts the user to enter a string and counts the number of vowels (a, e, i, o, u) in that string.

```python
# Challenge 6: Count Vowels in a String
string = input("Enter a string: ")

count = 0
for char in string:
    if char.lower() in 'aeiou':
        count += 1

print("Number of vowels:", count)
```

This program prompts the user to enter a string, iterates through each character in the string, checks if it's a vowel, and increments a counter if it is. Lastly, it displays the overall count of vowels.

Challenge 7: Check if a Number is Even or Odd

Write a Python program that prompts the user to enter a number and checks if it's even or odd.

```python
# Challenge 7: Check if a Number is Even or Odd
number = int(input("Enter a number: "))

if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

This program prompts the user to enter a number, checks if it's divisible by 2 (i.e., even), and prints the result accordingly.

Challenge 8: Find the Maximum of Two Numbers

Write a Python program that prompts the user to enter two numbers and finds the maximum of the two.

```python
# Challenge 8: Find the Maximum of Two Numbers
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

maximum = max(num1, num2)
print("Maximum:", maximum)
```

This program prompts the user to enter two numbers, uses the `max()` function to find the maximum of the two, and then prints the result.

Challenge 9: Check if a Number is Prime

Write a Python program that prompts the user to enter a number and checks if it's a prime number.

```python
# Challenge 9: Check if a Number is Prime
number = int(input("Enter a number: "))

if number > 1:
    for i in range(2, int(math.sqrt(number)) + 1):
        if number % i == 0:
                print("Not Prime")
                break
    else:
        print("Prime")
else:
    print("Not Prime")
```

This program prompts the user to enter a number, iterates through all numbers from 2 to the square root of the number, and checks if any of them divide the number evenly. If not, it's considered a prime number.

Challenge 10: Fibonacci Series

Write a Python program that prints the Fibonacci series up to a specified number of terms.

```python
# Challenge 10: Fibonacci Series
num_terms = int(input("Please provide the number of terms: "))

first_term = 0
second_term = 1

print("Fibonacci Series:")
for i in range(num_terms):
    print(first_term, end=" ")
    next_term = first_term + second_term
    first_term = second_term
    second_term = next_term
```

This program prompts the user to enter the number of terms in the Fibonacci series, initializes the first two terms as 0 and 1, and then iterates to generate the subsequent terms based on the sum of the previous two terms.

These challenges provide a solid foundation for working with numbers, strings, and user input in Python. By understanding these concepts and practicing them in coding challenges, beginners can gain confidence and proficiency in Python programming. Stay tuned for more challenges and guides as you continue your Python journey!

Section 2: Control Flow

In this section, we'll explore challenges 11-20, which focus on using conditional statements and loops in various scenarios. Control flow statements such as if/else and loops like for/while are crucial for controlling the flow of execution in a Python program. These challenges will help beginners understand how to use these constructs effectively to solve a variety of problems.

Challenge 11: Check Leap Year

Write a Python program that prompts the user to enter a year and checks if it's a leap year.

```python
# Challenge 11: Check Leap Year
year = int(input("Enter a year: "))

if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print("Leap Year")
else:
    print("Not a Leap Year")
```

This program prompts the user to enter a year, checks if it's divisible by 4 and not divisible by 100, or if it's divisible by 400. If either condition is true, it's considered a leap year.

Challenge 12: Print Multiplication Table

Write a Python program that prompts the user to enter a number and prints its multiplication table up to a specified range.

```python
# Challenge 12: Print Multiplication Table
number = int(input("Enter a number: "))
range_limit = int(input("Enter the range limit: "))

print("Multiplication Table for", number, ":")
for i in range(1, range_limit + 1):
    print(number, "x", i, "=", number * i)
```

This program prompts the user to enter a number and a range limit, then iterates from 1 to the range limit and prints the multiplication table for the given number.

Challenge 13: Check Palindrome

Write a Python program that prompts the user to enter a string and checks if it's a palindrome.

```python
# Challenge 13: Check Palindrome
string = input("Enter a string: ")

if string == string[::-1]:
    print("Palindrome")
else:
```

```
    print("Not a Palindrome")
```

This program prompts the user to enter a string, reverses the string using slicing, and then checks if the original string is equal to its reverse.

Challenge 14: Find Factorial

Write a Python program that prompts the user to enter a number and finds its factorial.

```python
# Challenge 14: Find Factorial
number = int(input("Enter a number: "))

factorial = 1
for i in range(1, number + 1):
    factorial *= i

print("Factorial:", factorial)
```

This program prompts the user to enter a number and calculates its factorial by multiplying all the numbers from 1 to the given number.

Challenge 15: Print Fibonacci Series

Write a Python program that prompts the user to enter the number of terms and prints the Fibonacci series.

```python
```

```
# Challenge 15: Print Fibonacci Series
num_terms = int(input("Enter the number of terms: "))

first_term = 0
second_term = 1

print("Fibonacci Series:")
for i in range(num_terms):
    print(first_term, end=" ")
    next_term = first_term + second_term
    first_term = second_term
    second_term = next_term
```

This program prompts the user to enter the number of terms in the Fibonacci series and prints the series up to that number of terms.

Challenge 16: Check Armstrong Number

Write a Python program that prompts the user to enter a number and checks if it's an Armstrong number.

```python
# Challenge 16: Check Armstrong Number
number = int(input("Enter a number: "))
original_number = number
num_digits = len(str(number))
sum = 0
```

```
while number > 0:
    digit = number % 10
    sum += digit ** num_digits
    number //= 10

if sum == original_number:
    print("Armstrong Number")
else:
    print("Not an Armstrong Number")
```

This program prompts the user to enter a number, calculates the sum of its digits raised to the power of the number of digits, and checks if it's equal to the original number.

Challenge 17: Find GCD

Write a Python program that prompts the user to enter two numbers and finds their greatest common divisor (GCD).

```python
# Challenge 17: Find GCD
import math

num1 = int(input("Enter the first number: "))
num2 = int(input("Enter the second number: "))

gcd = math.gcd(num1, num2)
print("GCD:", gcd)
```

```

This program prompts the user to enter two numbers and uses the `gcd()` function from the `math` module to find their greatest common divisor.

**Challenge 18: Reverse a Number**

Write a Python program that prompts the user to enter a number and prints its reverse.

```python
# Challenge 18: Reverse a Number
number = int(input("Enter a number: "))

reverse = 0
while number > 0:
    digit = number % 10
    The variable "reverse" is updated by multiplying its current value by 10 and then adding the value of "digit" to it.
    number //= 10

print("Reverse:", reverse)
```

This program prompts the user to enter a number, iteratively extracts the digits from the number, and builds the reverse number by appending each digit to the right of the current reverse. Finally, it prints the reverse of the input number.

Challenge 19: Print Pattern
```

Write a Python program that prompts the user to enter the number of rows and prints a pattern.

```python
# Challenge 19: Print Pattern
rows = int(input("Enter the number of rows: "))

for i in range(1, rows + 1):
    print("*" * i)
```

This program prompts the user to enter the number of rows and prints a pattern of asterisks (*), where the number of asterisks in each row increases by one from 1 to the specified number of rows.

Challenge 20: Check Prime Number

Write a Python program that prompts the user to enter a number and checks if it's a prime number.

```python
# Challenge 20: Check Prime Number
number = int(input("Enter a number: "))

if number > 1:
    Iterate through the range starting from 2 up to the square root of "number" plus 1.
        if number % i == 0:
                print("Not Prime")
                break
    else:
```

```
        print("Prime")
else:
    print("Not Prime")
```

This program prompts the user to enter a number, iterates from 2 to the square root of the number, and checks if any of the numbers divide the input number evenly. If not, the number is considered prime.

These challenges provide practice in using conditional statements and loops to solve various problems. By understanding and mastering these constructs, beginners can become proficient in controlling the flow of execution in their Python programs. Stay tuned for more challenges and guides as you continue your Python journey!

Section 3: Functions

In this section, we'll delve into challenges 21-30, which focus on defining and applying functions for code reusability. Functions are a fundamental aspect of programming that allow you to encapsulate a block of code and execute it multiple times with different inputs. By defining functions, you can modularize your code, improve readability, and promote code reuse. Let's explore these challenges and see how functions can be utilized effectively.

Challenge 21: Calculate Area of a Circle

Write a Python function that calculates the area of a circle given its radius.

```python
# Challenge 21: Calculate Area of a Circle
import math
```

```
def calculate_area(radius):
    return math.pi * radius ** 2

radius = float(input("Enter the radius of the circle: "))
print("Area of the circle:", calculate_area(radius))
```

In this challenge, we define a function `calculate_area()` that takes the radius of the circle as input and returns its area. We then prompt the user to enter the radius and call the function to calculate and print the area.

Challenge 22: Check Even or Odd

Create a Python function to determine whether a provided number is even or odd.

```python
# Challenge 22: Check Even or Odd
def check_even_odd(number):
    if number % 2 == 0:
        return "Even"
    else:
        return "Odd"

number = int(input("Enter a number: "))
print(check_even_odd(number))
```

Here, we define a function `check_even_odd()` that takes a number as input and returns "Even" if the number is even, and "Odd" otherwise. We then prompt the user to enter a number and call the function to check and print whether it's even or odd.

Challenge 23: Convert Celsius to Fahrenheit

Develop a Python function to convert a temperature from Celsius to Fahrenheit.

```python
# Challenge 23: Convert Celsius to Fahrenheit
def celsius_to_fahrenheit(celsius):
    return (celsius * 9/5) + 32

celsius = float(input("Please input the temperature in Celsius: "))
print("Temperature in Fahrenheit:", celsius_to_fahrenheit(celsius))
```

In this challenge, we define a function `celsius_to_fahrenheit()` that takes a temperature in Celsius as input and returns its equivalent in Fahrenheit. We prompt the user to enter the temperature in Celsius and call the function to convert and print the temperature in Fahrenheit.

Challenge 24: Check Palindrome

Create a Python function to verify whether a provided string is a palindrome.

```python
# Challenge 24: Check Palindrome
def check_palindrome(string):
```

```
    return string == string[::-1]

string = input("Enter a string: ")
if check_palindrome(string):
    print("Palindrome")
else:
    print("Not a Palindrome")
```

Here, we define a function `check_palindrome()` that takes a string as input and returns True if it's a palindrome (i.e., the same forwards and backwards), and False otherwise. We prompt the user to enter a string and call the function to check and print whether it's a palindrome.

Challenge 25: Calculate Factorial

Write a Python function that calculates the factorial of a given number.

```python
# Challenge 25: Calculate Factorial
def calculate_factorial(number):
    factorial = 1
    Iterate through the range starting from 1 up to and including "number".
        factorial *= i
    return factorial

number = int(input("Enter a number: "))
print("Factorial:", calculate_factorial(number))
```

```

In this challenge, we define a function `calculate_factorial()` that takes a number as input and returns its factorial. We prompt the user to enter a number and call the function to calculate and print its factorial.

**Challenge 26: Find GCD**

Write a Python function that finds the greatest common divisor (GCD) of two numbers.

```python
# Challenge 26: Find GCD
import math

def find_gcd(num1, num2):
    return math.gcd(num1, num2)

num1 = int(input("Enter the first number: "))
num2 = int(input("Enter the second number: "))
print("GCD:", find_gcd(num1, num2))
```

Here, we define a function `find_gcd()` that takes two numbers as input and returns their greatest common divisor using the `gcd()` function from the `math` module. We prompt the user to enter two numbers and call the function to find and print their GCD.

Challenge 27: Print Fibonacci Series

Write a Python function that prints the Fibonacci series up to a specified number of terms.
```

```python
# Challenge 27: Print Fibonacci Series
def fibonacci_series(num_terms):
    first_term, second_term = 0, 1
    for _ in range(num_terms):
        print(first_term, end=" ")
        next_term = first_term + second_term
        first_term = second_term
        second_term = next_term


num_terms = int(input("Please input the number of terms: "))
fibonacci_series(num_terms)
```

In this challenge, we define a function `fibonacci_series()` that takes the number of terms as input and prints the Fibonacci series up to that number of terms. We prompt the user to enter the number of terms and call the function to print the series.

Challenge 28: Reverse a String

Write a Python function that reverses a given string.

```python
# Challenge 28: Reverse a String
def reverse_string(string):
    return string[::-1]
```

```
string = input("Enter a string: ")
print("Reversed string:", reverse_string(string))
```

In this section, we establish a function called `reverse_string()` which accepts a string as an argument and yields its reverse through slicing. We prompt the user to enter a string and call the function to reverse and print the string.

Challenge 29: Check Armstrong Number

Create a Python function to determine whether a provided number is an Armstrong number.

```python
# Challenge 29: Check Armstrong Number
def check_armstrong(number):
    num_digits = len(str(number))
    sum = 0
    temp = number
    while temp > 0:
        digit = temp % 10
        sum += digit ** num_digits
        temp //= 10
    return sum == number

number = int(input("Enter a number: "))
if check_armstrong(number):
    print("Armstrong Number")
```

```
else:
    print("Not an Armstrong Number")
```

In this challenge, we define a function `check_armstrong()` that takes a number as input and returns True if it's an Armstrong number (i.e., the sum of its digits raised to the power of the number of digits is equal to the original number), and False otherwise. We prompt the user to enter a number and call the function to check and print whether it's an Armstrong number.

Challenge 30: Print Pattern

Write a Python function that prints a pattern based on the number of rows specified.

```python
# Challenge 30: Print Pattern
def print_pattern(rows):
    for i in range(1, rows + 1):
        print("*" * i)

rows = int(input("Enter the number of rows: "))
print_pattern(rows)
```

Here, we define a function `print_pattern()` that takes the number of rows as input and prints a pattern of asterisks (*), where the number of asterisks in each row increases by one from 1 to the specified number of rows. We prompt the user to enter the number of rows and call the function to print the pattern.

These challenges demonstrate the power and versatility of functions in Python programming. By defining and applying functions effectively, you can modularize your code, improve its readability, and promote code reusability. As you continue your journey in Python programming, mastering functions will be essential for writing efficient and maintainable code. Stay tuned for more challenges and guides as you enhance your Python skills!